

Regression with the Partial Ordered Stereotype Model

Laia Egea-Cortes

Introduction

The `stereord` package provides tools for modeling ordinal response variables using the newly developed Partial Ordered Stereotype Model (POSM). Additionally, it includes model selection procedures specifically designed for the POSM.

In order to demonstrate the use of the `posm()` function, we will first load the `arthritis` dataset from the `multgee` package (Touloumis, A., 2015). This dataset contains self-assessment scores from 302 rheumatoid arthritis patients at three follow-up times (variable `time` recorded in months).

```
library(MASS)
library(stereord)
library(multgee)
data(arthritis)
head(arthritis)
#>   id y sex age trt baseline time
#> 1  1 4  2  54  2      2      1
#> 2  1 5  2  54  2      2      3
#> 3  1 5  2  54  2      2      5
#> 4  2 4  1  41  1      3      1
#> 5  2 4  1  41  1      3      3
#> 6  2 4  1  41  1      3      5
```

The variable `y` is the self-assessment score, measured on a five-level ordinal response scale from 1 (very poor) to 5 (very good), and will serve as our ordinal response. We will transform it into a factor, which required for the analysis. Since the response is already coded with values from 1 to 5, we do not need to modify the default factor levels.

```
arthritis$y <- factor(arthritis$y)
```

Since the POSM is designed for cross-sectional data, and the `arthritis` dataset is longitudinal, we will select the first follow-up data for each subject (`time = 1`) to ensure the data structure is appropriate for the analysis.

```
arthritis <- arthritis[arthritis$time == 1,]
```

We will transform the covariates `sex` and `trt` into factors:

```
arthritis$sex <- factor(arthritis$sex)
arthritis$trt <- factor(arthritis$trt)
```

Model Fitting

We will now fit the Partial Ordered Stereotype Model, whose selection will be motivated by model selection procedures discussed in the following section. The model includes three covariates: baseline, trt, and sex, grouped into two groups: baseline and sex in one group, and trt in another.

If we define Y as our ordinal response variable with q levels indexed by $k = 2, \dots, q$, and the response is dependent on the covariates baseline defined as x_1 , trt as x_2 and sex as x_3 . Therefore, the partial ordered stereotype model takes the form

$$\log \left(\frac{P(Y = k | \mathbf{x})}{P(Y = 1 | \mathbf{x})} \right) = \alpha_k + \phi_k^1(\beta_1 x_1 + \beta_2 x_2) + \phi_k^2(\beta_3 x_3)$$

where α_1 is fixed at 0 for identifiability and $0 = \phi_1^h \leq \dots \leq \phi_k^h \leq \dots \leq \phi_q^h = 1$ for each $h = 1, 2$. The restriction on the ϕ_k^h values is what makes this the partial **ordered** stereotype model.

The β values are the coefficients of the covariates, as in a linear regression model, except here they change the probability of Y taking different levels. If a single coefficient is positive, it increases the probability of Y taking higher levels, and if it is negative it increases the probability of Y taking lower levels, as x increases.

Note that this is **not** a cumulative model, nor is it a logistic model, because the log function on the left hand side of the model equation is not a logit.

To specify the grouping when fitting the model using the `posm()` function, we use the `grouping` argument. We assign the same number (1) to the positions corresponding to baseline and sex (first and third), indicating they belong to the same group, and a different number (2) to trt, as it forms a separate group.

```
fit <- posm(formula = y ~ baseline + trt + sex, grouping = c(1,2,1),
            data = arthritis)

fit
#> Call:
#> posm(formula = y ~ baseline + trt + sex, grouping = c(1, 2, 1),
#>       data = arthritis)
#>
#> Coefficients beta:
#> baseline      trt2      sex2
#> 1.9320889 1.8429039 0.7689313
#>
#> Intercepts alpha:
#>           1/2           1/3           1/4           1/5
#> 0.0000000 1.2148841 0.2715167 -2.5456164 -6.8370898
#>
#> Score parameters phi:
#>      Group 1  Group 2
#> 1 0.00000000 0.0000000
#> 2 0.02714079 0.3886305
#> 3 0.27569323 1.0000000
#> 4 0.67596817 1.0000000
#> 5 1.00000000 1.0000000
#>
#> Grouping:
#> [1] "Group 1 : baseline, sex2" "Group 2 : trt2"
#>
#> Residual Deviance: 702.9672
#> AIC: 728.9672
#> BIC: 777.073
```

```
#> (3 observations deleted due to missingness)
#> Warning: did not converge as iteration limit reached
```

Notice that the output of this initial fit includes a warning message: Warning: did not converge as iteration limit reached. This warning indicates that the `optim()` function inside `posm()`, which is used to fit the model, did not manage to converge to a solution.

We therefore need to refit the model allowing `optim()` to run for more iterations, to ensure convergence. We can do this by passing in the `control` argument for `optim()`, which is a list of control parameters for `optim()`. The only control parameter we will change is the upper limit on the number of iterations, `maxit`. The default is 100, and we will increase it to 5000.

```
fit <- posm(formula = y ~ baseline + trt + sex, grouping = c(1,2,1),
            data = arthritis, control=list(maxit=5000))

fit
#> Call:
#> posm(formula = y ~ baseline + trt + sex, grouping = c(1, 2, 1),
#>       data = arthritis, control = list(maxit = 5000))
#>
#> Coefficients beta:
#> baseline      trt2      sex2
#> 1.8983831 1.8571162 0.7599929
#>
#> Intercepts alpha:
#>           1/2           1/3           1/4           1/5
#> 0.0000000 1.3264803 0.3640261 -2.4549192 -6.7523372
#>
#> Score parameters phi:
#>      Group 1  Group 2
#> 1 0.000000000 0.0000000
#> 2 0.005787626 0.3927153
#> 3 0.262280241 1.0000000
#> 4 0.669810529 1.0000000
#> 5 1.000000000 1.0000000
#>
#> Grouping:
#> [1] "Group 1 : baseline, sex2" "Group 2 : trt2"
#>
#> Residual Deviance: 702.9357
#> AIC: 728.9357
#> BIC: 777.0415
#> (3 observations deleted due to missingness)
```

Now we have obtained a converged solution.

We can show a summary using the standard `summary()` function:

```
s.fit <- summary(fit)
#>
#> Re-fitting to get Hessian
s.fit
#> Call:
#> posm(formula = y ~ baseline + trt + sex, grouping = c(1, 2, 1),
#>       data = arthritis, control = list(maxit = 5000))
```

```

#>
#> Coefficients (beta):
#>           Value Std. Error t value Pr(>|t|)
#> baseline  1.8984      0.4528   4.193 2.76e-05 ***
#> trt2       1.8571      0.7979   2.328  0.0199 *
#> sex2       0.7600      0.5222   1.455  0.1456
#>
#> Intercepts (alpha):
#>           Value Std. Error t value Pr(>|t|)
#> 1|2   1.3265      0.7996   1.659  0.0971 .
#> 1|3   0.3640      0.8677   0.420  0.6748
#> 1|4  -2.4549      0.9726  -2.524  0.0116 *
#> 1|5  -6.7523      1.6303  -4.142 3.45e-05 ***
#>
#> Score parameters (phi):
#>           Value Std. Error
#> phi2|Group 1 0.005788  1.385e-01
#> phi3|Group 1 0.262280  1.260e-01
#> phi4|Group 1 0.669811  1.407e-01
#> phi2|Group 2 0.392715  3.049e-01
#> phi3|Group 2 1.000000  1.083e-04
#> phi4|Group 2 1.000000  7.824e-06
#>
#> Test Score parameters (phi):
#>           t value Pr(>|t|)
#> Group 1: phi1 vs phi2   0.042  0.96666
#> Group 1: phi2 vs phi3 -2.364  0.01810 *
#> Group 1: phi3 vs phi4 -3.263  0.00110 **
#> Group 1: phi4 vs phi5 -2.346  0.01897 *
#> Group 2: phi1 vs phi2   3.117  0.00182 **
#> Group 2: phi2 vs phi3 -4.862 1.16e-06 ***
#> Group 2: phi3 vs phi4   0.000  1.00000
#> Group 2: phi4 vs phi5   0.000  1.00000
#>
#> Grouping:
#> [1] "Group 1 : baseline, sex2" "Group 2 : trt2"
#>
#> Residual Deviance: 702.9357
#> AIC: 728.9357
#> BIC: 777.0415
#> (3 observations deleted due to missingness)

```

Applying the `summary()` function to the output, we obtain the standard errors and test statistics. The standard errors for α_k and β are calculated as the square root of the diagonal elements of the inverse of the Fisher information matrix. Since the coefficients are estimated using the maximum likelihood method with a quasi-Newton approach, the Fisher information matrix corresponds to the Hessian. Therefore, the inverse of the Hessian serves as the variance-covariance matrix. As we estimate reparametrized sets of score parameters, the Delta method is applied to compute their standard errors. The statistical tests performed are t-tests, which assess whether each coefficient is significantly different from zero in the case of α_k and β . In the case of the score parameters, for each set, the tests assess whether each score parameter is significantly different from the previous or the subsequent score parameter.

We can now move on to interpreting the results.

Covariate coefficients (β)

The `beta` estimates are the estimated coefficients of the covariates `baseline`, `trt`, and `sex`. More positive values correspond to a higher probability of getting high response levels (i.e., in this case, you're more likely to get $y = 4$ or 5 than 1 or 2), and more negative values correspond to a higher probability of getting low response levels (i.e. you're more likely to get $y = 1$ or 2 than 4 or 5), as x increases.

We can access the covariate coefficients (β) from the model object using:

```
fit$beta
#> baseline      trt2      sex2
#> 1.8983831 1.8571162 0.7599929
```

We see that the coefficient for `baseline` is `1.90`, which indicates that the baseline score for an individual has a big impact on their follow-up self assessment score. If the baseline increases by one level, then that makes it more likely that the follow-up self-assessment score, y , will also be higher.

We see that the coefficient for `trt` is `1.86`, similar to the one for `baseline`. The reference level `1`, corresponds to placebo group, and `2` corresponds to drug group. So drug group patients are more likely to have high self-assessment scores than placebo group patients.

Finally, we see that the coefficient for `sex` is `0.76`. The reference level, `1`, corresponds to female, and `2` corresponds to male. So male patients are more likely to have high self-assessment scores than female patients.

Overall, as shown in the summary, the effect sizes suggest that `baseline` and `trt` are relevant, as both are significantly different of `0`, whereas `sex` does not appear to be particularly important.

Intercepts (α_k)

The `alpha` estimates in the output are, roughly speaking, the intercept terms for each level of the response. This is why there is no intercept term amongst the covariate coefficients, it is already incorporated into the `alpha` parameters. The intercept parameters determine the proportion of individuals per response category at $x = 0$. We do not need to interpret these estimates.

The intercepts (α_k) can be accessed from the model object using:

```
fit$alpha
#>
#>      1/2      1/3      1/4      1/5
#> 0.0000000 1.3264803 0.3640261 -2.4549192 -6.7523372
```

Score parameters (ϕ_k)

The `phi` estimates in the output correspond to the score parameters. Each set of score parameters captures the discriminant ability of the covariates in a given group across the response categories. For each group, the first and last values are fixed at `0` and `1`, respectively.

We can access the score parameters (ϕ_k) from the model object using:

```
fit$phi
#>      Group 1      Group 2
#> 1 0.00000000 0.0000000
#> 2 0.005787626 0.3927153
#> 3 0.262280241 1.0000000
#> 4 0.669810529 1.0000000
#> 5 1.000000000 1.0000000
```

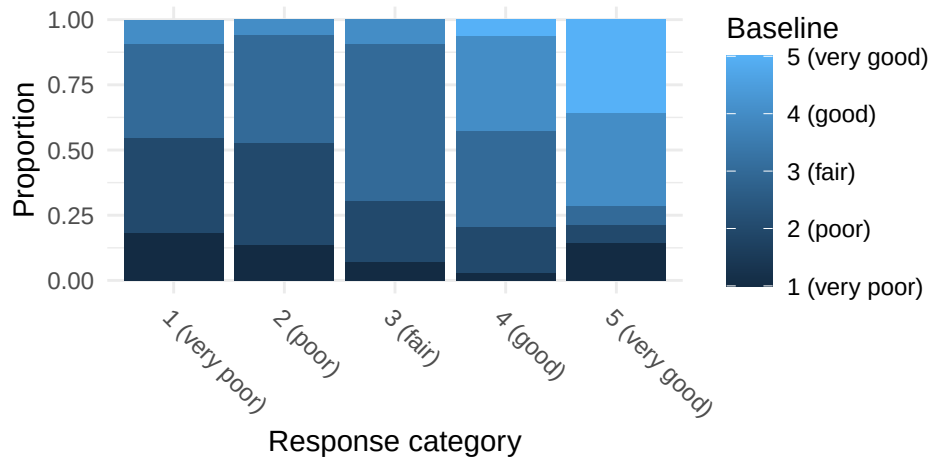
Each column corresponds to a full set of score parameters for one covariate group.

For the first group, composed of the covariates baseline and sex, the estimated scores are $\hat{\phi}_2^1 = 0.01$, $\hat{\phi}_3^1 = 0.26$, and $\hat{\phi}_4^1 = 0.67$. Since $\hat{\phi}_2^1$ is very close to the fixed value $\phi_1^1 = 0$, this suggests that baseline and sex have limited discrimination between levels 1 (very poor) and 2 (poor) of the response. However, as the subsequent ϕ_k^1 values increase progressively, it indicates that these covariates provide better discrimination among higher response levels.

This pattern can be visualised by examining the distribution of baseline values across the response categories:

```
library(dplyr)
library(ggplot2)
labels.y <- c(
  "1" = "1 (very poor)",
  "2" = "2 (poor)",
  "3" = "3 (fair)",
  "4" = "4 (good)",
  "5" = "5 (very good)"
)

arthritis %>%
  filter(complete.cases(.)) %>%
  group_by(y, baseline) %>%
  summarise(count = n(), .groups = "drop") %>%
  group_by(y) %>%
  mutate(proportion = count / sum(count)) %>%
  ggplot(aes(x = factor(y), y = proportion, fill = baseline)) +
  geom_bar(stat = "identity") +
  scale_x_discrete(labels = labels.y) +
  scale_fill_continuous(
    breaks = as.numeric(names(labels.y)),
    labels = labels.y,
    name = "Baseline"
  ) +
  ylab("Proportion") +
  xlab("Response category") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = -45, hjust = 0))
```



From the plot, we can observe that the baseline values are similarly distributed across categories 1 (very poor) and 2 (poor), but show increasingly distinct patterns for categories 3 (fair), 4 (good), and 5 (very good). These interpretations are further supported by the statistical tests for the score parameters. The test comparing ϕ_1^1 and ϕ_2^1 is not significant (p-value = 0.967), confirming that baseline and sex have limited discrimination between the first two response levels. In contrast, the tests for higher levels show significant differences (p-values < 0.05), indicating discriminant ability among higher response categories.

For the second group, composed solely of the covariate trt, the estimated scores are $\hat{\phi}_2^2 = 0.39$, and $\hat{\phi}_3^2 = \hat{\phi}_4^2 = 1.00$. In this case, the treatment covariate is able to discriminate between the lower response levels but shows limited discrimination between levels 3 (fair), 4 (good), and 5 (very good), as $\hat{\phi}_3^2$, $\hat{\phi}_4^2$, and ϕ_5^2 are all equal to 1. Similar as in group 1, significant differences are found between ϕ_1^2 and ϕ_2^2 and between ϕ_2^2 and ϕ_3^2 (p-value < 0.01), while no significant differences are observed between ϕ_3^2 , ϕ_4^2 , and ϕ_5^2 (p-value = 1), confirming the limited discrimination among the highest response levels for the treatment covariate.

Note that, given the estimated score parameters in the second group, we could consider refitting the POSM by fixing $\phi_3^2 = \phi_4^2 = 1$, which would simplify the model without compromising interpretability and may improve parameter stability and estimation efficiency. Similarly, fixing $\phi_2^1 = 0$ in group 1 could be considered for the same reason. If the p-value for a comparison between a pair of score parameters is above 0.05, this does not necessarily imply that the corresponding score parameters must be fixed to the same value. Rather, it suggests that fixing the score parameters for those levels to the same value would not compromise the model or its interpretation, as the covariates in that particular group exhibit limited discriminant ability for those specific response categories, and would continue to do so after being fixed to the same value.

Odds ratios

In the Partial Ordered Stereotype Model, a single odds ratio cannot be computed for each covariate coefficient. This is because the model is purposely constructed so that the covariates can have slightly different effects on the different levels of the response variable. The overall effect of a covariate is determined jointly by its coefficient and the corresponding score parameters ϕ_k for each response level k .

As a result, there is no single number, such as a traditional odds ratio, that captures the overall impact of a covariate. Instead, we obtain a different odds ratio for each comparison between a response category and the baseline category, resulting in $q-1$ odds ratios, where q is the number of response categories. These odds ratios are computed as the exponential of the product of the covariate coefficient and the corresponding score parameter, that is, $\exp(\beta \cdot \phi_k)$.

For the baseline covariate:

```
round(exp(fit$beta[1]*fit$phi[,1]), 3)
#>      1      2      3      4      5
#> 1.000 1.011 1.645 3.566 6.675
```

For the trt covariate:

```
round(exp(fit$beta[2]*fit$phi[,2]), 3)
#>      1      2      3      4      5
#> 1.000 2.074 6.405 6.405 6.405
```

And for the sex covariate:

```
round(exp(fit$beta[3]*fit$phi[,1]), 3)
#>      1      2      3      4      5
#> 1.000 1.004 1.221 1.664 2.138
```

Note that we multiply each covariate coefficient by the corresponding set of score parameters (the first set for the baseline and sex covariates, and the second set for the trt covariate).

For example, the interpretation of the odds ratios for the baseline covariate is as follows. The first odds ratio (1.011) indicates that a one-unit increase in baseline leads to only a very small change in the odds of being in category 2 versus category 1. For category 3 versus 1, the odds increase by approximately 64.5%. The odds of being in category 4 rather than category 1 are about 3.57 times greater, and for category 5 versus 1, about 6.68 times greater, per unit increase in baseline.

Model Selection

The data set includes four potential covariates related to 'y': sex (coded as 1 for female and 2 for male), age (recorded at baseline), trt (treatment group variable, coded as 1 for the placebo group and 2 for the drug group), and baseline (self-assessment score of rheumatoid arthritis at baseline).

To identify the optimal model, defined as the candidate model with the lowest AIC value, we apply three selection strategies: one based on the forward stepwise algorithm, another based on the combined forward-backward stepwise algorithm, and a large-scale systematic search. These methods will select the relevant covariates, number of groups of covariates, and grouping the covariates accordingly.

Forward Stepwise

A model selection strategy for the POSM that builds the model incrementally using a forward stepwise approach. At each step, the method selects both the covariate and its group assignment that achieve the greatest improvement in the Akaike Information Criterion (AIC). Therefore, this algorithm not only selects the relevant predictors, but also determines the appropriate number of covariate groups and assigns predictors to groups accordingly.

We run the forward stepwise method:

```
fs.arthritis <- forward.stepwise.POSM(Outcome = "y",
                                     Covariates = c("sex", "age", "trt",
                                                    "baseline"),
                                     data = arthritis)

fs.arthritis
#> [1] "y ~ baseline + trt + sex (grouping: 1, 2, 1; AIC: 728.967) Time: 28.038 secs"
```

The output includes the selected model, the grouping and AIC (in parentheses), and the time spent to run the algorithm and reach the optimal model, in seconds.

By default, this method uses the AIC as the stopping rule (IC = "AIC"). Additionally, it place all dummy variables generated from a categorical variable into the same group (split_categorical = FALSE). Finally, it does not show every intermediate step (keep = FALSE).

If we instead use the BIC as the information criterion, the result changes:

```
forward.stepwise.POSM(Outcome = "y",
                      Covariates = c("sex", "age", "trt", "baseline"),
                      data = arthritis, IC = "BIC")
#> [1] "y ~ baseline (SetCov: 1; BIC: 767.851) Time: 21.71 secs"
```

If we would like to see all the steps taken during the model selection process:


```

forward.stepwise.POSM(Outcome = "y",
                      Covariates = c("sex", "age", "trt", "baseline"),
                      data = arthritis, keep = TRUE)

#>
#> "y ~ baseline (grouping: 1; AIC: 738.247) Time: 6.403 secs"
#>
#> "y ~ baseline + trt (grouping: 1, 2; AIC: 729.063) Time: 27.95 secs"
#>
#> "y ~ baseline + trt + sex (grouping: 1, 2, 1; AIC: 728.967) Time: 64.997 secs"

```

Note that three steps were needed to find the optimal model.

Forward-Backward Stepwise

The forward-backward stepwise method provides a more flexible approach than forward selection, allowing both forward additions and backward eliminations of covariates during the model building process.

We run the forward-backward stepwise method:

```

fbs.arthritis <- forward.backward.stepwise.POSM(Outcome = "y",
                                                Covariates = c("sex", "age",
                                                                "trt",
                                                                "baseline"),
                                                data = arthritis)

fbs.arthritis
#> [1] "y ~ baseline + trt + sex (grouping: 1, 2, 1; AIC: 728.967) Time: 29.075 secs"

```

As with the `forward.stepwise.POSM()` function, this method uses the AIC as the stopping rule. Note that the option to use the BIC is not yet implemented in this function. Like before, all dummy variables generated from a categorical variable are placed into the same group (`split_categorical = FALSE`), and intermediate steps are not shown by default (`keep = FALSE`). Additionally, this function includes a `maxSteps` argument to define the maximum number of steps allowed during the selection process, with a default value of 100.

Large-Scale Systematic Search

This model selection method performs a large-scale systematic search of the overall best model. It fits all possible models for each number of possible covariates in each step until no further improvement in AIC is observed.

We run the large-scale systematic search method:

```

lsss.arthritis <- large.scale.systematic.search.POSM(Outcome = "y",
                                                    Covariates = c("sex", "age",
                                                                    "trt", "baseline"),
                                                    data = arthritis)

lsss.arthritis
#> [1] "y ~ sex + trt + baseline (SetCov: 1, 2, 1; AIC: 728.967) Time: 83.219 secs"

```

As the `forward.backward.stepwise.POSM()` function, this method uses the AIC as the stopping rule and the option to use the BIC is not yet implemented. Like before, all dummy variables generated from a

categorical variable are placed into the same group (`split_categorical = FALSE`), and intermediate steps are not shown by default (`keep = FALSE`).

Note that all methods arrived at the same optimal model. The FBS method required slightly more time than the FS method because, in addition to fitting models that add covariates, it also fits models that remove covariates. The large-scale systematic search method takes significantly more time to run, as it fits all candidate models with a given number of covariates at each step. The number of candidate models increases sharply as the number of covariates grows. For this reason, computational time should be carefully considered when the number of potential covariates is large.

References

Touloumis, A. (2015). R package multgee: a generalized estimating equations solver for multinomial responses. *Journal of Statistical Software*, 64, 1-14.